Implement AI to solve a simple turn-based game (Tic-Tac-Toe)

Name : Asmit Sahu Roll : 23052231

In [1]:
```
from collections import deque
import heapq
import copy
```

In [2]:
```
# Board representation :: X = AI ; O = Opponent ; _ = Empty
board = [
    ['X', 'O', '_'],
    ['_', 'X', '_'],
    ['O', '_', '_']
]
def print_board(b):
    for row in b:
        print(" ".join(row))
    print()
```

In [3]:
```
def check_winner(board): # Utility Function!
    lines = []
    # Rows and Columns
    for i in range(3):
        lines.append(board[i])  # row
        lines.append([board[0][i], board[1][i], board[2][i]])  # column
    # Diagonals
    lines.append([board[0][0], board[1][1], board[2][2]])
    lines.append([board[0][2], board[1][1], board[2][0]])
    for line in lines:
        if line == ['X','X','X']:
            return 'X'
        if line == ['O','O','O']:
            return 'O'
    # Draw
    if all(cell != '_' for row in board for cell in row):
        return 'Draw'
    return None

def get_moves(board):
    moves = []
    for i in range(3):
        for j in range(3):
            if board[i][j] == '_':
                moves.append((i, j))
    return moves
```

In [4]:
```
def bfs(board):
    queue = deque([(board, True)])  # True = X's turn
    nodes = 0
```

```
    while queue:
        state, x_turn = queue.popleft()
        nodes += 1
        winner = check_winner(state)
        if winner:
            return winner, nodes
        for move in get_moves(state):
            new_state = copy.deepcopy(state)
            new_state[move[0]][move[1]] = 'X' if x_turn else 'O'
            queue.append((new_state, not x_turn))

    return None, nodes
```

In [5]:
```
def dfs(board):
    stack = [(board, True)]
    nodes = 0
    while stack:
        state, x_turn = stack.pop()
        nodes += 1
        winner = check_winner(state)
        if winner:
            return winner, nodes
        for move in get_moves(state):
            new_state = copy.deepcopy(state)
            new_state[move[0]][move[1]] = 'X' if x_turn else 'O'
            stack.append((new_state, not x_turn))

    return None, nodes
```

In [6]:
```
def heuristic(board):
    winner = check_winner(board)
    if winner == 'X':
        return 10
    if winner == 'O':
        return -10
    return 0
```

In [7]:
```
def astar(board):
    pq = []
    heapq.heappush(pq, (-heuristic(board), 0, board, True))
    nodes = 0
    while pq:
        _, cost, state, x_turn = heapq.heappop(pq)
        nodes += 1
        winner = check_winner(state)
        if winner:
            return winner, nodes
        for move in get_moves(state):
            new_state = copy.deepcopy(state)
            new_state[move[0]][move[1]] = 'X' if x_turn else 'O'
            h = heuristic(new_state)
            heapq.heappush(pq, (-h, cost+1, new_state, not x_turn))

    return None, nodes
```

In [8]:
```
bfs_result, bfs_nodes = bfs(board)
dfs_result, dfs_nodes = dfs(board)
astar_result, astar_nodes = astar(board)

print("BFS Result:", bfs_result, "; Nodes explored:", bfs_nodes)
print("DFS Result:", dfs_result, "; Nodes explored:", dfs_nodes)
print("A* Result: ", astar_result, "; Nodes explored:", astar_nodes)
```

```
BFS Result: X ; Nodes explored: 6
DFS Result: X ; Nodes explored: 2
A* Result:  X ; Nodes explored: 2
```